

Tekst zadatka – Industrial Processing System API

Implementirati thread-safe servis **ProcessingSystem** koji simulira obradu industrijskih zadataka (Jobs). Sistem treba da podrži **thread-safe** konkurentnu obradu, asinhrono izvršavanje i event-driven arhitekturu.

- Kreirati klasu **Job** sa sledećim poljima:
 - *Guid Id*
 - *JobType*¹ *Type*
 - *string Payload*
 - *int Priority*²
- Kreirati klasu **JobHandle** koja predstavlja rezultat izvršavanja:
 - *Guid Id*
 - *Task<int> Result*
- Implementirati klasu **ProcessingSystem** koja:
 - Prima poslove putem metode *Submit(Job job)*
 - Vraća *JobHandle*
 - Obradu poslova vrši asinhrono koristeći *Task*

¹dodati enum

²manji broj znači veći prioritet

Konfiguracija sistema

- Sistem se inicijalizuje iz priloženog *XML* konfiguracionog fajla. Za *payload* smatrati da je uvek u istom i validnom formatu.
- Na osnovu konfiguracije:
 - kreirati odgovarajući broj worker niti
 - inicijalno učitati poslove iz XML-a
- Poslovi sa većim prioritetom moraju biti obrađeni pre poslova sa manjim prioritetom.
- MaxQueueSize predstavlja maksimalan broj Job-ova u sistemu, odbijaju se novi poslovi ukoliko je queue popunjen.

- Implementirati obradu različitih tipova poslova³:
 - **Prime** – izračunavanje broja prostih brojeva do zadate vrednosti (paralelno, proslediti broj niti koje će vršiti obradu). Ograničiti broj niti prilikom parsiranja na interval [1,8].
 - **IO** – simulacija čitanja stanja na određenoj adresi korišćenjem *Thread.Sleep*, vraća se nasumičan broj između 0 i 100.
- Implementirati **idempotentnost**:
 - isti Job (isti Id) ne sme biti izvršen više puta

³Payload je ulazni parametar i potrebno ga je parsirati, za Prime predstavlja do koje vrednosti se računaju brojevi i koliko niti računa a za IO koliko kašnjenje se simulira (u ms)

Event sistem i testiranje

- Definisati događaje:
 - *JobCompleted*
 - *JobFailed*
- Na događaje se pretplatiti koristeći **lambda izraze**.
- Svaki događaj mora asinhrono upisati u fajl:

[DateTime] [Status] JobId, Result
- *Job* je *failed* ako mu treba duže od dve sekunde da se izvrši. Ukoliko dođe do *fail*-a nekog *Job*-a pokušati *retry* dva puta. Ukoliko i treći put *Job fail*-uje dodati *ABORT* u log fajl za taj Job i ignorisati njegov rezultat.

Dodatne metode i izveštaj

- Implementirati metodu:

```
IEnumerable<Job> GetTopJobs(int n)
```

- vratiti prvih N poslova po prioritetu iz trenutno aktivnog reda

- Implementirati metodu:

```
Job GetJob(Guid id)
```

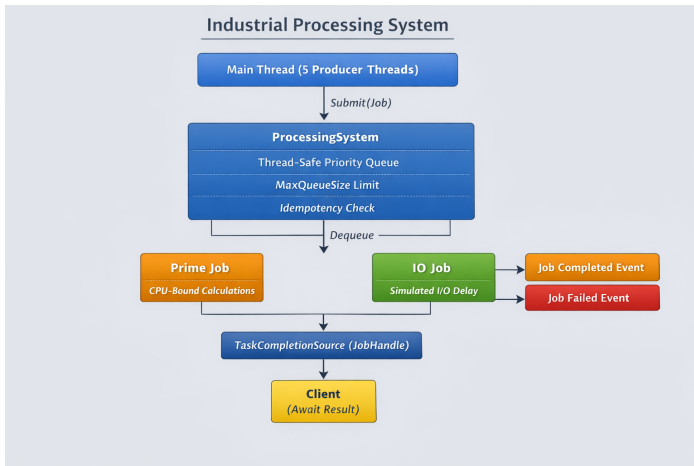
- vratiti objekat za zadat *ID*

- Na svakih minut generisati izveštaj korišćenjem **LINQ**:
 - broj izvršenih poslova po tipu
 - prosečna vreme izvršavanja posla po tipu
 - broj neuspešnih poslova, grupisanih (sortiranih) po tipu
- Izveštaj upisati u XML fajl. Omogućiti da se poslednjih 10 izveštaja čuvaju u posebnim fajlovima. Ukoliko se generiše novi izveštaj nakon desetog, on će prepisati najstariji.

Main program

- U glavnom programu:
 - pročitati broj niti iz konfiguracionog fajla
 - pokrenuti odgovarajući broj niti koje nasumično dodaju nove poslove
 - svaka nit nasumično dodaje nove poslove
- Obezbediti:
 - thread-safe pristup sistemu
 - da red ne pređe maksimalnu veličinu iz konfiguracije
- Implementirati odgovarajuće **try-catch** blokove.

Dijagram



Zadatak zamisliti kao producer-consumer, samo asinhrono i sa prioritetima.

- Radi lakše implementacije moguće je dodati određena polja i metode u klase ali one koje su navede moraju da postoje za maksimalan broj poena.
- Potrebno je implemenitrati unit testove i postići *line code coverage* od barem 67%.
- Kolokvijum se brani uživo u četvrtak (7.5) na ličnim laptopovima, uz demonstraciju rada i usmeno propitivanje i diskusiju o implementaciji.
- Tačno vreme i mesto odbrane će biti objavljeni u narednim danima.
- Poslati link od GitHub repozitorijuma asistentu do srede u 23:59, neka na linku bude neki vid potvrde *code coverage*-a (slika/podešeno automatsko izvršavanje testova)